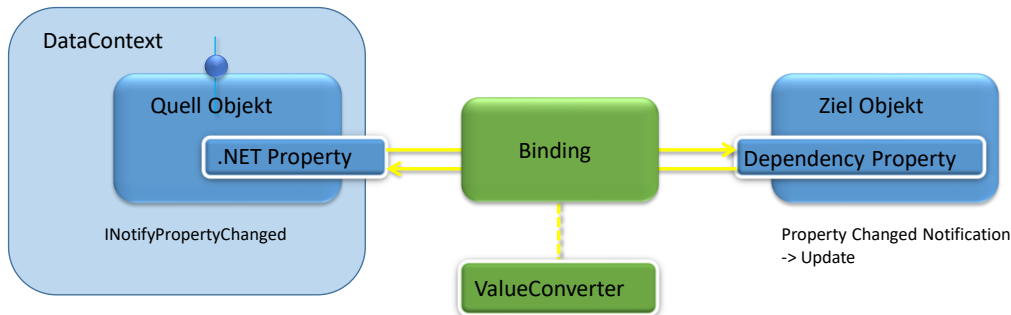


DataBinding

DataBinding

- Binding ermöglicht den Datenzugriff sowie Datenänderung
- Daten werden vom DataContext abgerufen
- .NET-Properties benötigen *INotifyPropertyChanged* um mit der Oberfläche zu kommunizieren



DataBinding ermöglicht die automatische Synchronisierung von Daten zwischen der Benutzeroberfläche und Datenquellen. Es ermöglicht die dynamische Aktualisierung von UI-Elementen basierend auf Änderungen der zugrunde liegenden Daten. Durch die Bindung von Eigenschaften eines UI-Elements an Eigenschaften eines Datenobjekts kann das UI reaktiv auf Änderungen der Daten reagieren, ohne dass Code-behind erforderlich ist.

Die verwendete Binding-Klasse ist die gleich, wie beim PropertyBinding, wveshalb auch die meisten Funktionen und Eigenschaften (wie Mode, UpdateSourceTrrgger, etc.) auf das DataBinding übertragbar sind. Auch hier können Converter-Klassen zwischengeschaltet werden.

Die entsprechenn Datenquellen können jedes beliebige instanzierte Objekt sein. Dieses wird in die DataContext-Eigenschaft des entsprechenden umgebenden Controls eingefügt (vgl. nächste Seite). Damit die einzelnen Eigenschaften des Quellobjekts mit der GUI kommunizieren können, muss über das Interface *INotifyPropertyChanged* eine Aktualisierungsmechanik implementiert werden.

DataContext

- *DataContext*-Property (in Controls und Containern)
 - Kann beliebige Objekte beinhalten, an deren Properties angebunden werden kann
 - Markup-Extension-Kurzbindungen: `{Binding Propertyname}`

```
<StackPanel x:Name="Spl_DataContextBsp">
    <StackPanel.DataContext>
        <local:Person Vorname="Anna" Nachname="Meier" Alter="29"/>
    </StackPanel.DataContext>

    <TextBlock Text="{Binding Path=Vorname}"/>
    <TextBlock Text="{Binding Nachname}"/>
    <TextBlock Text="{Binding Alter}"/>
</StackPanel>
```

Jedes Window- und Containerobjekt besitzt die object-Eigenschaft ‚DataContext‘. Hiermit kann für den entsprechenden Bereich ein Quellobjekt für das DataBinding festgelegt werden. Die Anbindung erfolgt aus diesem Bereich heraus über ein Binding-Objekt, bei welchem der ‚ElementName‘ nicht angegeben wird. Dies führt zu einer Verwendung des DataBindings.

Der Datentyp des Context-Objekts ist den Umständen entsprechend frei wählbar. Unter MVVM (siehe entsprechender Abschnitt) wird hier ein sogenanntes ViewModel verwendet, welches eine Klasse ist, die unter anderem Model-Daten bündelt und für die Oberfläche aufbereitet.

INotifyPropertyChanged

- Interface zur Kommunikation von .Net-Properties mit DependencyProperties

```
public class Person : INotifyPropertyChanged
{
    private string vorname;
    public string Vorname
    {
        get => vorname;
        set
        {
            vorname = value;
            OnPropertyChanged(nameof(Vorname));
        }
    }

    public event PropertyChangedEventHandler? PropertyChanged;

    public void OnPropertyChanged(string propertyName)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
    }
}
```

Das `INotifyPropertyChanged`-Interface ist ein wichtiger Bestandteil des `DataBinding`-Konzepts. Es ermöglicht eine Benachrichtigung über Änderungen an den Eigenschaften eines Objekts, damit das UI entsprechend aktualisiert werden kann. Um das `INotifyPropertyChanged`-Interface zu verwenden, muss eine Klasse es implementieren und ein Ereignis namens `PropertyChanged` bereitstellen. Dieses Ereignis wird ausgelöst, wenn sich der Wert einer Eigenschaft ändert bzw. wenn die GUI auf die Veränderung reagieren soll.

Der Implementierungsprozess beinhaltet das Hinzufügen einer Methode, die das `PropertyChanged`-Ereignis auslöst. Diese Methode wird normalerweise `OnPropertyChanged` genannt und wird aufgerufen, wenn eine Eigenschaft geändert wird.

In der `OnPropertyChanged`-Methode wird das `PropertyChanged`-Ereignis ausgelöst und der Name der geänderten Eigenschaft als Argument übergeben. Dies ermöglicht es dem UI, zu erkennen, welche Eigenschaft aktualisiert werden muss.

In der Klasse, die das `INotifyPropertyChanged`-Interface implementiert, sollten die Eigenschaften als öffentliche Eigenschaften mit `get`- und `set`-Zugriffen deklariert werden. Im Setter der Eigenschaft wird die `OnPropertyChanged`-Methode aufgerufen, um das `PropertyChanged`-Ereignis auszulösen.

`INotifyPropertyChanged` ermöglicht eine reaktive Datenbindung in WPF, bei der das UI automatisch aktualisiert wird, wenn sich die Daten ändern. Dies fördert eine effiziente und dynamische Aktualisierung des Benutzerinterfaces und erleichtert die Entwicklung von ansprechenden und interaktiven Anwendungen.

ObservableCollection<T>

- Listen-Klasse mit Funktionen von List<T>
- Integrierter Benachrichtigungsmechanismus
 - Informiert die GUI automatisch über die Liste betreffende Veränderungen wie
 - Hinzufügen oder Löschen eines Elements
 - Veränderung der Reihenfolge
 - Informiert die GUI **nicht** über Veränderungen innerhalb der beinhaltenden Elemente

Die Klasse ObservableCollection<T> bietet eine dynamische Auflistung von Objekten des angegebenen Typs T, vergleichbar mit List<T>, die zusätzlich automatisch Benachrichtigungen über Änderungen an der Auflistung sendet.

Der Hauptzweck der ObservableCollection<T> besteht darin, das UI über Änderungen in der Auflistung zu informieren, damit es entsprechend aktualisiert werden kann. Wenn Objekte zur Auflistung hinzugefügt, daraus entfernt oder geändert werden, löst die ObservableCollection<T> Ereignisse wie CollectionChanged und PropertyChanged aus, auf die die GUI reagiert. Es ist somit kein zusätzlicher Einsatz von INotifyPropertyChanged nötig.

Durch die Verwendung der ObservableCollection<T> können Entwickler das Datenmodell und das UI besser entkoppeln. Sie müssen nicht manuell Änderungsereignisse auslösen oder das UI aktualisieren, da die ObservableCollection<T> diese Aufgaben übernimmt. Darüber hinaus bietet die ObservableCollection<T> auch Methoden wie Add, Remove, Clear usw., um die Verwaltung der Auflistung zu erleichtern. Diese Methoden lösen automatisch die entsprechenden Ereignisse aus und sorgen für eine konsistente Aktualisierung des UIs.